

PRE-CHANGE REVIEW — BILLING & WEBHOOK PATHS

Blast radius: what this change touches, and what is merely claimed

Produced by the blast-radius reviewer agent. Every finding carries file:line. Guards are reported as verified, wired-untested, or claimed-only — a control counts as done only with a call site on the live path and a test that exercises it.

TARGET	SCOPE	RUNBOOK	REVIEW TIME
EXAMPLES/LEGACY-SHOP	BILLING + WEBHOOK ROUTES	THIRD-RAIL:HARDENING-RUNBOOK	4-5 MINUTES

§ 01 — AFFECTED ROUTES AND CHAINS

```
GET /api/billing/invoices → express.json → requireAuth → handler
POST /api/billing/charge → express.json → requireAuth → requireAdmin → handler
POST /api/billing/refund → express.json → handler ← no guards of any kind
POST /api/webhooks/payments → express.json → handler ← no guards of any kind
```

Body parsing: the global `express.json()` at `server.js:12` is the only parser in the app, mounted before both router mounts (`server.js:20-21`). No `express.raw` exists anywhere, so the webhook handler never sees the raw bytes it needs for signature verification.

§ 02 — FINDINGS THE RUNBOOK COVERS · ITEMS APPLIED: 1, 2, 3, 4, 5, 7, 9

BLOCKER Failed webhook signature is logged, processed anyway, and acked 200

`routes/webhooks.js:22-32` · runbook item 3

WHY Anyone POSTing `{"type":"payment.succeeded"}` gets fulfillment to run and the provider told it was delivered — forgery with no retry path, live since the 2022 TODO.

FIX Return 400 on mismatch and stop; handler failures return 5xx so the provider retries.

BLOCKER POST /refund carries zero middleware while both siblings require auth

`routes/billing.js:16` · runbook item 5

WHY An unauthenticated request can ship a refund for any `orderId` and amount. "Added in a hurry during the 2024 holiday incident" — the comment says so itself.

FIX One line: `router.post('/refund', requireAuth, requireAdmin, ...)` — the chain its sibling `/charge` already carries.

BLOCKER Signature verified against re-serialized JSON — raw bytes already destroyed

server.js:12, routes/webhooks.js:14-16 · runbook item 1

WHY Verification of legitimate events fails intermittently on key order, whitespace, unicode — the failure mode that bred the skip-and-continue TODO. Unreliable as built.

FIX Mount `express.raw({ type: 'application/json' })` on the `/payments` route; verify the raw buffer; parse only after verification.

WARN The org's tested constant-time verifier has zero live call sites

utils/verifySignature.js:5 · runbook item 2

WHY The 2023-security-push helper is correct, tested, and ready — and protects nothing. Its passing unit tests are evidence it works, zero evidence it runs. The fix that never happened.

HOW IT HAPPENS The code's own comments tell it: one engineer wrote the proper verifier in the "2023 security push"; another had already hand-rolled the inline check under a 2022 TODO. Two efforts, two years, never connected — every dashboard green the whole time.

FIX Call `verifySignature({...})` in the `/payments` handler; delete the inline HMAC block.

WARN No idempotency — provider redelivery re-runs fulfillment

routes/webhooks.js:28-30 · runbook item 4

WHY Providers redeliver. A redelivered `payment.succeeded` double-fulfills once that branch is real.

FIX Record processed event ids before the fulfillment branch; no-op on replay.

WARN Webhook endpoint has no rate limiter of its own

routes/webhooks.js:10 · runbook item 7

WHY The far less sensitive `/api/search` carries one (`server.js:15`); the unauthenticated POST that triggers processing work does not.

FIX Dedicated limiter tier on the `/payments` chain.

WARN WEBHOOK_SECRET falls back to a committed demo value; boot never refuses

routes/webhooks.js:6 · runbook item 9

WHY The moment enforcement is fixed, an unset production env var silently verifies against `whsec_demo_secret` — public in this repo — keeping forgery trivial with no error anywhere.

FIX Remove the fallback; production boot exits if the secret is missing or short.

§ 03 — FINDINGS THE RUNBOOK DOES NOT COVER · CANDIDATES FOR THE ORG'S RUNBOOK IF THEY RECUR

WARN Refund handler forwards unvalidated attacker-controlled orderId and amount

routes/billing.js:17-18

WHY Even after the auth fix lands, any admin request — or stolen admin token — can refund an arbitrary orderId for an arbitrary amount, negative values included. No existence, ownership, or cap check.

NOT IN RUNBOOK Item 5 covers guard coverage and ordering; nothing covers validating a refund payload against the original charge.

INFO Inline signature check is weaker than the shelved util on three axes

routes/webhooks.js:16-22 vs utils/verifySignature.js:8-21

WHY !== string comparison instead of timingSafeEqual, no timestamp-freshness window, no length check. Once mismatches stop being skipped, these become the live weaknesses.

NOT IN RUNBOOK Items 1-3 mandate raw bytes, a wired verifier, fail-closed behavior; none prescribes constant-time comparison for webhook signatures.

§ 04 — GUARDS: CLAIMED VS VERIFIED · THE DENOMINATOR THE SCOREBOARD COUNTS

GUARD	STATE	EVIDENCE
requireAuth	WIRED, UNTESTED	call sites routes/billing.js:7, :11 · only suite is test/verifySignature.test.js, which exercises no route
requireAdmin	WIRED, UNTESTED	call site routes/billing.js:11 · no test exercises the route
verifySignature	CLAIMED ONLY	defined utils/verifySignature.js:5, zero live call sites · a test file is never a call site

Every dashboard would say webhooks are verified here. This table says nothing is.

§ 05 — ADJACENT, NOT REVIEWED

— routes/auth.js — listed in .third-rail.json sensitivePaths, outside this review's blast radius. It gets its own review when someone edits it.

§ 06 — WHAT THIS REVIEW COULD NOT VERIFY STATICALLY

- No tests were executed; coverage judged by reading test/. No route-level path is exercised anywhere — which is why no guard reaches "verified."
- The route map is a regex scan: dynamic registration or spread-array middleware would be invisible. None observed in the 8 scanned files, which is not proof of absence.
- The fulfillment branch is a stub comment; downstream idempotency is not visible from this repo.
- Whether production sets WEBHOOK_SECRET, and what a proxy in front supplies, is deployment configuration not present here.

§ 07 — SCOREBOARD

COUNTS ONLY · RECONCILES AGAINST THE SECTIONS ABOVE

```
Runbook findings:  7 (items 1, 2, 3, 4, 5, 7, 9)  3 BLOCKER, 4 WARN, 0 INFO
Beyond the runbook: 2                                0 BLOCKER, 1 WARN, 1 INFO
Guards:           0 verified, 2 wired untested, 1 claimed only (of 3)
Blast radius:     4 routes, 4 files
```

Caller: reproduce both findings sections and this scoreboard verbatim for the reader; commentary goes after, not instead.